

Indexing in C++





Indexing in C++





**Indexing
in C++**

**Custom
Hash Index**

Indexing in Database Systems

INDEX	
Klein, Arno	249
La Duke, Elliott W.	377
Lawless, Frank R.	307
Layer, John G.	330
Leffel, John C.	357
Leffel, Edward	359
Leonard, Frederick P.	371
Lewis, Edward	338
Lewis, Frank E.	290
Lewis, James R.	327
Lewis, Thompson P.	339
Llewelyn, Edgar J.	331
....., George T.	334
.....ermann, Francis B.	298
.....din, Enoch W.	387
.....din, James F.	389
.....din, Noah	389
McReynolds, Samuel M.	296
MacGregor, Francis B.	244
Macy, Carlos B.	287
Marvel, Alexander L.	400
Marvel, Thomas	397
Meinschein, Conrad	338
Menzies, G. V.	217
Menzies, Winston	218
Miller, Lorenz C.	385
Moeller, John H.	256
Montgomery, Samuel B.	282
Morrow, Lannie G.	350
Moye, James H.	362

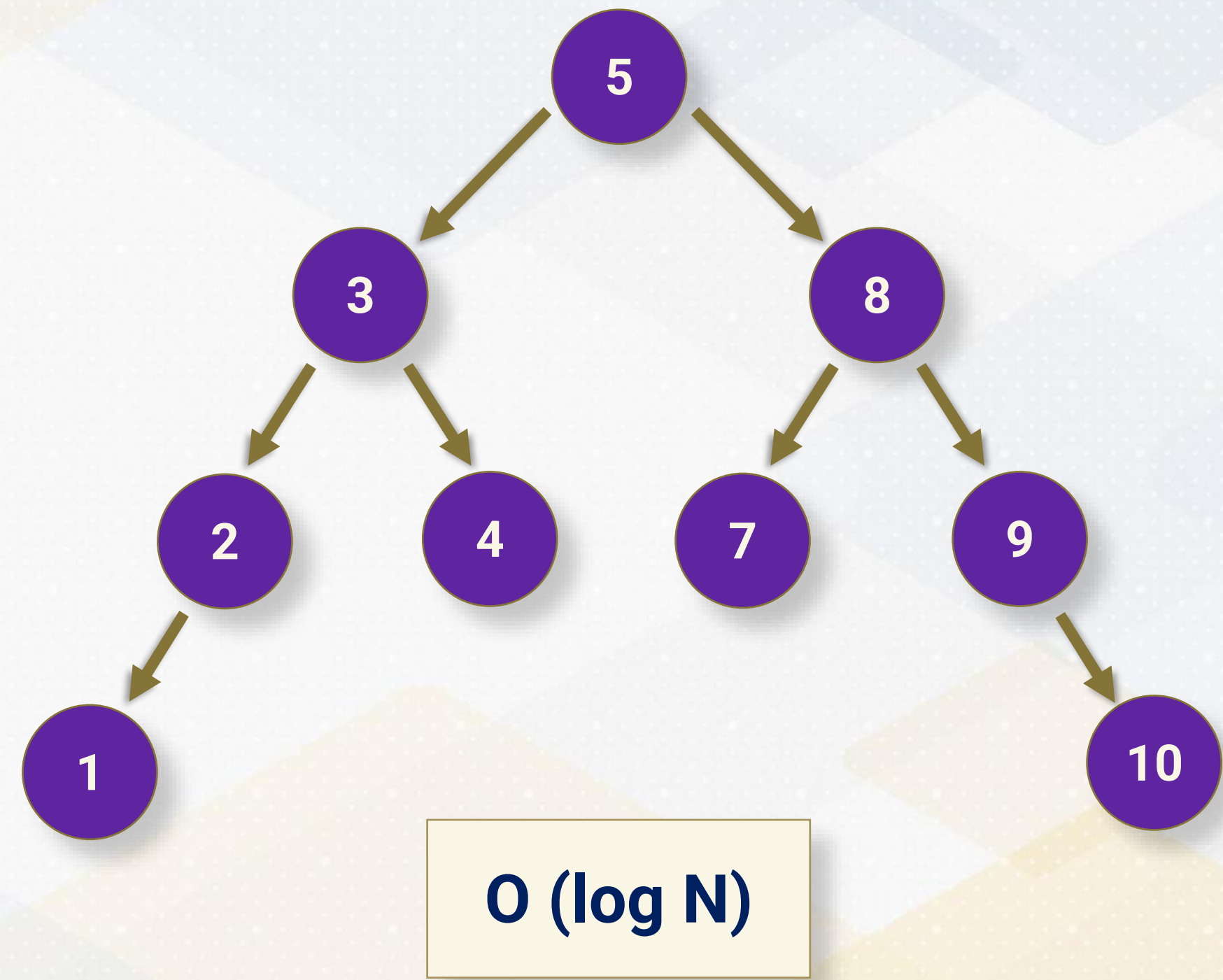
Book Index

Index Finger



Indexing in C++: ordered_map

Key	Value
1	Apple
2	Banana
3	Cherry
4	Date
5	Elderberry
6	Fig
7	Grape
8	Honeydew
9	Ilama
10	Jackfruit



Indexing in C++: unordered_map

Key	Value
2	Banana
6	Fig
1	Apple
3	Cherry
4	Date
5	Elderberry
8	Honeydew
7	Grape
10	Jackfruit
9	Ilama

Avg Case

$O(1)$

Worst Case

$O(N)$

map vs unordered_map

	MAP	UNORDERED MAP
Ordering	Sorted based on key	Not sorted based on key
Implementation	Binary Search Tree	Hash Table
Average Case Time	$O(\log N)$	$O(1)$
Worst Case Time	$O(\log N)$	$O(N)$

HashIndex in BuzzDB

```
class HashIndex {  
private:  
    std::unordered_map<int, int> hash_index;  
public:  
    void insertOrUpdate(int key, int value) {  
        hash_index[key] = value;  
    }  
    int getValue(int key) const {  
        auto it = hash_index.find(key);  
        return it != hash_index.end() ? it->second : -1;  
    }  
};
```

HashIndex in BuzzDB

insertOrUpdate

```
class HashIndex {  
private:  
    std::unordered_map<int, int> hash_index;  
public:  
    void insertOrUpdate(int key, int value) {  
        hash_index[key] = value;  
    }  
    int getValue(int key) const {  
        auto it = hash_index.find(key);  
        return it != hash_index.end() ? it->second : -1;  
    }  
};
```

HashIndex in BuzzDB

insertOrUpdate

getValue

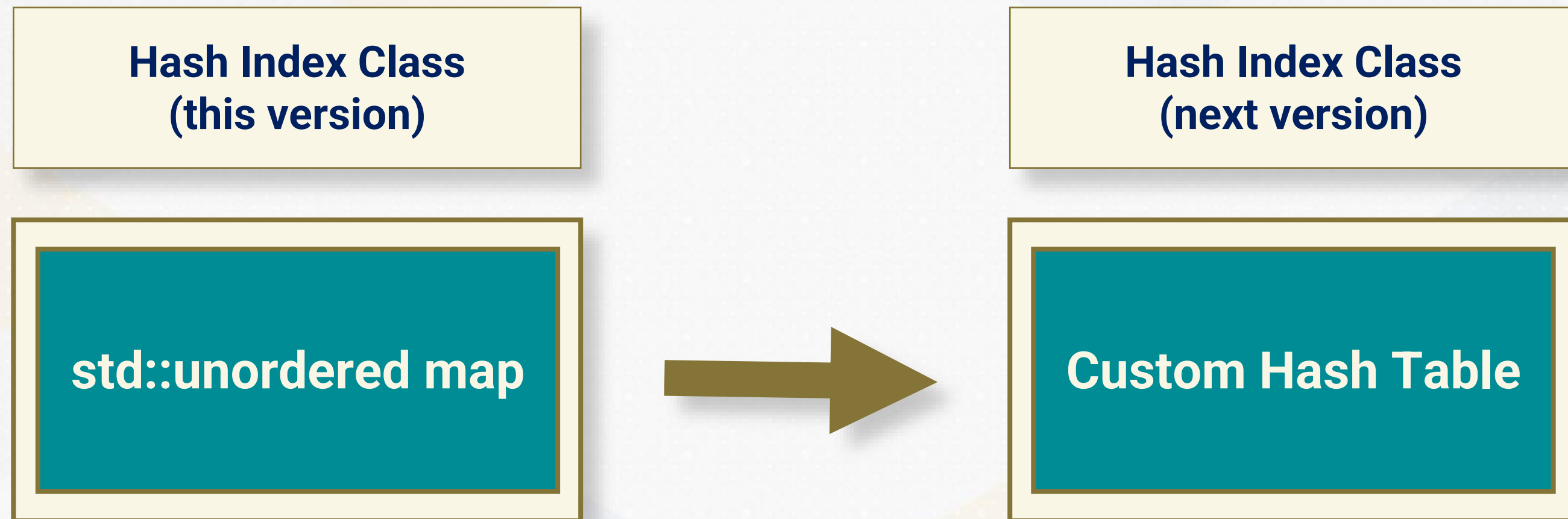
```
class HashIndex {  
private:  
    std::unordered_map<int, int> hash_index;  
public:  
    void insertOrUpdate(int key, int value) {  
        hash_index[key] = value;  
    }  
    int getValue(int key) const {  
        auto it = hash_index.find(key);  
        return it != hash_index.end() ? it->second : -1;  
    }  
};
```

Integration of HashIndex in BuzzDB

During the scanning phase, each tuple's key-value pair is inserted into the HashIndex.

```
void BuzzDB::scanTableToBuildIndex() {  
    // Iterating over pages and tuples  
    for (size_t page_itr = 0; page_itr < num_pages; page_itr++) {  
        // Extract key-value pairs and insert them into the index  
        int key = loadedTuple->fields[0]->asInt();  
        int value = loadedTuple->fields[1]->asInt();  
  
        index.insertOrUpdate(key, value);  
    }  
}
```

Custom HashTable







Indexing in C++





**Indexing
in C++**

**Custom
Hash Index**

Hash Table



Hash Table



Hash Table



**Hash
Functions**



Hash Table



**Hash
Functions**

**Collision
Handling**

Hash Table

Think of a **hash table** like a **cabinet of drawers**.
Each drawer can hold a piece of paper with the **key** and **associated value**.

0	1	2	3	4	5	6	7	8	9
					15				
					Apple				

DIRECT ACCESS USING HASH FUNCTION

Hash Table

Think of a **hash table** like a **cabinet of drawers**.
Each drawer can hold a piece of paper with the **key** and **associated value**.

0	1	2	3	4	5	6	7	8	9
					15				
					Apple				



DIRECT ACCESS USING HASH FUNCTION

Hash Table

Think of a **hash table** like a **cabinet of drawers**.
Each drawer can hold a piece of paper with the **key** and **associated value**.

0	1	2	3	4	5	6	7	8	9
					15				
					Apple				



DIRECT ACCESS USING HASH FUNCTION

Inserting Key-Value Pair

Insert (15, Apple)

0	1	2	3	4	5	6	7	8	9
					15				
					Apple				

$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 15 \% 10 = 5$ (since $15 = 10 * 1 + 5$)

Inserting Key-Value Pair

Insert (15, Apple)

0	1	2	3	4	5	6	7	8	9
					15				
					Apple				



$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 15 \% 10 = 5$ (since $15 = 10 * 1 + 5$)

Inserting Key-Value Pair

Insert (26, Grape)

0	1	2	3	4	5	6	7	8	9
					15	26			
					Apple	Grape			

HASH(KEY) = KEY % 10 = 26 % 10 = 6 (since $26 = 10 * 2 + 6$)

Inserting Key-Value Pair

Insert (26, Grape)

0	1	2	3	4	5	6	7	8	9
					15	26			
					Apple	Grape			



$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 26 \% 10 = 6$ (since $26 = 10 * 2 + 6$)

Inserting Key-Value Pair with Collision

Insert (56, Kiwi)

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 56 \% 10 = 6$$

Inserting Key-Value Pair with Collision

Insert (56, Kiwi)

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 56 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 15 = Apple

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 15 \% 10 = 5$$

Finding Key-Value Pair

Find value associated with key 15 = Apple

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 15 \% 10 = 5$$

Finding Key-Value Pair

Find value associated with key 56 = Kiwi

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 56 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 56 = Kiwi

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 56 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 56 = Kiwi

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 56 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 86 = KEY NOT FOUND

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 86 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 86 = KEY NOT FOUND

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 86 \% 10 = 6$$

Finding Key-Value Pair

Find value associated with key 86 = KEY NOT FOUND

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 86 \% 10 = 6$$

Hash Function

Hash Function

Hash function maps keys to slots in the hash table

```
size_t hashFunction(int key) {  
    return key % totalSlots; // Our simple formula  
}
```

$$\text{HASH}(\text{KEY}) = \text{KEY} \% \text{TABLE_SIZE}$$

Collision Handling

Linear Probing

Move to the next available slot until we find one empty.

0	1	2	3	4	5	6	7	8	9
80	11				15	26	56	18	79
Lime	Date				Apple	Grape	Kiwi	Fig	Pear

If slot 8 is taken, check slot 9, then 0, 1, and so on.

Collision Handling

Linear Probing

Move to the next available slot until we find one empty.

0	1	2	3	4	5	6	7	8	9
80	11				15	26	56	18	79
Lime	Date				Apple	Grape	Kiwi	Fig	Pear



If slot 8 is taken, check slot 9, then 0, 1, and so on.

Collision Handling

Linear Probing

Move to the next available slot until we find one empty.

0	1	2	3	4	5	6	7	8	9
80	11				15	26	56	18	79
Lime	Date				Apple	Grape	Kiwi	Fig	Pear



If slot 8 is taken, check slot 9, then 0, 1, and so on.

"Linear" Probing

Formula to find the index of the next slot I after K “probes” is
“linear” with respect to K.

$$\text{INDEX } I = (\text{HASH}(\text{KEY}) + K) \% \text{TABLE_SIZE}$$

$$\text{HASH}(\text{KEY}) = \text{KEY} \% \text{TABLE_SIZE}$$

HashEntry Struct

HashEntry Struct

Represents a piece of paper containing **key** and **value**.

```
struct HashEntry {  
    int key, value;  
    HashEntry(int k, int v) : key(k), value(v) {}  
};
```

HashIndex

HashTable is simply an array of drawers. Each drawer can either hold a **HashEntry** or be **empty** (indicating no value).

```
class HashIndex {  
    std::vector<std::optional<HashEntry>> hashTable;  
  
    static const size_t capacity = 100; // Hard-coded capacity  
    HashIndex() { hashTable.resize(capacity); }  
};
```

Inserting Key-Value Pair

insertOrUpdate

Uses linear probing to find the next available slot for insertion or locate an existing key for updating.

```
void insertOrUpdate(int key, int value) {
    size_t index = hashFunction(key);
    do {
        if (!hashTable[index]) {
            hashTable[index] = HashEntry{key, value, true}; // Insert new entry
            break;
        } else if (hashTable[index]->key == key) {
            hashTable[index]->value += value; // Update existing entry
            break;
        }
        index = (index + 1) % capacity; // Linear probing to next slot
    } while (index != originalIndex); // Returned to starting point
}
```

Finding Key-Value Pair

getValue

The retrieval process employs linear probing as well to navigate through potential collision sequences.

```
int getValue(int key) const {  
    size_t index = hashFunction(key);  
    do {  
        if (hashTable[index] && hashTable[index]->key == key) {  
            return hashTable[index]->value; // Key found  
        }  
        index = (index + 1) % capacity; // Continue probing  
    } while (index != originalIndex);  
    return -1; // Key not found  
}
```

Operation Complexity

N = Number of key-value pairs in the hash table.

	NO/FEW COLLISIONS	LOTS OF COLLISIONS
INSERT	$O(1)$	$O(N)$
FIND	$O(1)$	$O(N)$

Clustering Problem with High Collision Rates

Bad Hash Function: $\text{HASH}(\text{KEY}) = 5$

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		

$\text{HASH}(\text{KEY}) = 5$

Clustering Problem with High Collision Rates

Bad Hash Function: $\text{HASH}(\text{KEY}) = 5$

0	1	2	3	4	5	6	7	8	9
					15	26	56		
					Apple	Grape	Kiwi		



$\text{HASH}(\text{KEY}) = 5$



Hash Table



Hash Table



**Hash
Functions**



Hash Table



**Hash
Functions**

**Collision
Handling**



Hash Table Position Tracking







Key Deletion



**Key
Deletion**



**Position
Tracking**

Finding key 32

Find value associated with key 32 = Fig

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Finding key 32

Find value associated with key 32 = Fig

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Finding key 32

Find value associated with key 32 = Fig

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Deletion

Delete key 13

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 13 \% 10 = 3$$

Deletion

Delete key 13

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 13 \% 10 = 3$$

Deletion

Delete key 13

0	1	2	3	4	5	6	7	8	9
		2		34	32				
		Apple		Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 13 \% 10 = 3$$

Finding key 32 After Deletion

Find value associated with key 32 = Not Found

0	1	2	3	4	5	6	7	8	9
		2		34	32				
		Apple		Kiwi	Fig				

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Finding key 32 After Deletion

Find value associated with key 32 = Not Found

0	1	2	3	4	5	6	7	8	9
		2		34	32				
		Apple		Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Finding key 32 After Deletion

Find value associated with key 32 = Not Found

0	1	2	3	4	5	6	7	8	9
		2		34	32				
		Apple		Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Solution #1: Rehashing All Keys

Insert all keys into another empty hash table.

0	1	2	3	4	5	6	7	8	9
		2	32	34					
		Apple	Fig	Kiwi					

$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$

Solution #2: Lazy Deletion

Mark a hash entry as deleted without removing it.

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				
1	1	1	0	1	1	1	1	1	1

Finding key 32 After Deletion

Find value associated with key 32 = Fig

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				
1	1	1	0	1	1	1	1	1	1



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Finding key 32 After Deletion

Find value associated with key 32 = Fig

0	1	2	3	4	5	6	7	8	9
		2	13	34	32				
		Apple	Grape	Kiwi	Fig				
1	1	1	0	1	1	1	1	1	1



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10 = 32 \% 10 = 2$$

Lazy Deletion using Exists Flag

```
struct HashEntry {  
    int key;  
    int value;  
    bool exists; // Indicates if the entry is active or deleted  
  
    // Updated constructor includes position  
    HashEntry(int k, int v, int pos) : key(k), value(v), exists(true) {}  
};
```

Vector to Static-Sized Array

```
class HashIndex {  
private:  
    static const size_t capacity = 100; // Hard-coded capacity  
    HashEntry hashTable[capacity];    // Static-sized array  
};
```

Position Tracking

Position is used for tracking the location of each entry within the hash table.

```
struct HashEntry {  
    int key;  
    int value;  
    int position; // Tracks the final position in the array  
    bool exists; // Indicates if the entry is active  
  
    // Updated constructor includes position  
    HashEntry(int k, int v, int pos)  
        : key(k), value(v), position(pos), exists(true) {}  
};
```

Position Tracking

```
void insertOrUpdate(int key, int value) {  
    size_t originalIndex = hashFunction(key);  
    bool inserted = false;  
    do {  
        if (!hashTable[index].exists) {  
            hashTable[index] = HashEntry(key, value, true);  
            hashTable[index].position = index;  
            inserted = true;  
            break;  
        }  
        ...  
    }  
}
```

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				



$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				

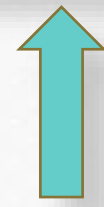


$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				

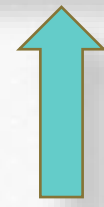


$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$

Limitations of Linear Probing

Clustering increases likelihood of collision; searches become inefficient

0	1	2	3	4	5	6	7	8	9
		2	12	22	32				
		Apple	Grape	Kiwi	Fig				



$$\text{HASH}(\text{KEY}) = \text{KEY} \% 10$$





Key Deletion



**Key
Deletion**

**Position
Tracking**



Double Hashing







Quadratic Probing



**Quadratic
Probing**

**Double
Hashing**

Quadratic Probing

Formula to find the index of the next slot I after K “probes” is “quadratic” with respect to K .

$$\text{INDEX } I = (\text{HASH}(\text{KEY}) + K^2) \% \text{TABLE_SIZE}$$

$$\text{HASH}(\text{KEY}) = \text{KEY} \% \text{TABLE_SIZE}$$

Inserting 2

Insert (2, Apple)

0	1	2	3	4	5	6	7	8	9
		2							
		Apple							



$$\text{HASH}(2) = 2; \text{K} = 0; \text{INDEX } I = (2 + 0^2) \% 10 = 2$$

Inserting 12

Insert (12, Grape)

0	1	2	3	4	5	6	7	8	9
		2	12						
		Apple	Grape						



$$\text{HASH}(12) = 2; K = 1; \text{INDEX } I = (2 + 1^2) \% 10 = 3$$

Inserting 12

Insert (12, Grape)

0	1	2	3	4	5	6	7	8	9
		2	12						
		Apple	Grape						



$$\text{HASH}(12) = 2; K = 1; \text{INDEX } I = (2 + 1^2) \% 10 = 3$$

Inserting 22

Insert (22, Kiwi)

0	1	2	3	4	5	6	7	8	9
		2	12			22			
		Apple	Grape			Kiwi			



$$\text{HASH}(22) = 2; K = 2; \text{INDEX } I = (2 + 2^2) \% 10 = 16 \% 10 = 6$$

Inserting 22

Insert (22, Kiwi)

0	1	2	3	4	5	6	7	8	9
		2	12			22			
		Apple	Grape			Kiwi			



$$\text{HASH}(22) = 2; K = 2; \text{INDEX } I = (2 + 2^2) \% 10 = 16 \% 10 = 6$$

Inserting 32

Insert (32, Fig)

0	1	2	3	4	5	6	7	8	9
	32	2	12			22			
	Fig	Apple	Grape			Kiwi			



$$\text{HASH}(32) = 2; K = 3; \text{INDEX } I = (2 + 3^2) \% 10 = 11 \% 10 = 1$$

Inserting 32

Insert (32, Fig)

0	1	2	3	4	5	6	7	8	9
	32	2	12			22			
	Fig	Apple	Grape			Kiwi			



$$\text{HASH}(32) = 2; K = 3; \text{INDEX } I = (2 + 3^2) \% 10 = 11 \% 10 = 1$$

Inserting 32

Insert (32, Fig)

0	1	2	3	4	5	6	7	8	9
	32	2	12			22			
	Fig	Apple	Grape			Kiwi			



$$\text{HASH}(32) = 2; K = 3; \text{INDEX } I = (2 + 3^2) \% 10 = 11 \% 10 = 1$$

Quadratic Probing

Probe sequence follows a quadratic formula.

```
void insertOrUpdate(int key, int value) {
    size_t originalIndex = hashFunction(key);
    bool inserted = false;
    int i = 0; // Attempt counter
    do {
        if (!hashTable[index].exists) {
            ...
        } else if (hashTable[index].key == key) {
            ...
        }
        i++;
        index = (originalIndex + i * i) % capacity; // Quadratic probing
    } while (index != originalIndex && !inserted);
}
```

Benefits of Quadratic Probing

Better spread of keys; higher search efficiency

0	1	2	3	4	5	6	7	8	9
	32	2	12			22			
	Fig	Apple	Grape			Kiwi			

Limitation of Quadratic Probing

Secondary clustering of all keys hashed to 2.

0	1	2	3	4	5	6	7	8	9
	32	2	12			22			
	Fig	Apple	Grape			Kiwi			



Double Hashing



Double Hashing

Double hashing

Uses a secondary hash function to calculate the probe step, offering a unique probe sequence for each key.

$$\text{INDEX } i = (\text{HASH1}(\text{KEY}) + K * \text{HASH2}(\text{KEY})) \% \text{TABLE_SIZE}$$

$$\text{HASH1}(\text{KEY}) = \text{KEY} \% \text{TABLE_SIZE}$$

$$\text{HASH2}(\text{KEY}) = 1 + \text{KEY} \% (\text{TABLE_SIZE} - 1)$$

Inserting 2

Insert (2, Apple)

0	1	2	3	4	5	6	7	8	9
		2							
		Apple							

HASH1(2) = 2; HASH2(2) = 1 + (2 % 9) = 1 + 2 = 3
K = 0; INDEX I = (2 + 0 * 3) % 10 = 2

Inserting 2

Insert (2, Apple)

0	1	2	3	4	5	6	7	8	9
		2							
		Apple							



HASH1(2) = 2; HASH2(2) = 1 + (2 % 9) = 1 + 2 = 3
K = 0; INDEX I = (2 + 0 * 3) % 10 = 2

Inserting 12

Insert (12, Grape)

0	1	2	3	4	5	6	7	8	9
		2				12			
		Apple				Grape			



HASH1(12) = 2; **HASH2**(12) = $1 + (12 \% 9) = 1 + 3 = 4$

K = 1; **INDEX I** = $(2 + 1 * 4) \% 10 = 6 \% 10 = 6$

Inserting 12

Insert (12, Grape)

0	1	2	3	4	5	6	7	8	9
		2				12			
		Apple				Grape			



$\text{HASH1}(12) = 2; \text{HASH2}(12) = 1 + (12 \% 9) = 1 + 3 = 4$
 $K = 1; \text{INDEX } I = (2 + 1 * 4) \% 10 = 6 \% 10 = 6$

Inserting 22

Insert (22, Kiwi)

0	1	2	3	4	5	6	7	8	9
		2				12	22		
		Apple				Grape	Kiwi		



HASH1(22) = 2; HASH2(22) = 1 + (22 % 9) = 1 + 4 = 5

K = 1; INDEX I = (2 + 1 * 5) % 10 = 7 % 10 = 7

Inserting 22

Insert (22, Kiwi)

0	1	2	3	4	5	6	7	8	9
		2				12	22		
		Apple				Grape	Kiwi		



HASH1(22) = 2; HASH2(22) = 1 + (22 % 9) = 1 + 4 = 5
K = 1; INDEX I = (2 + 1 * 5) % 10 = 7 % 10 = 7

Inserting 32

Insert (32, Fig)

0	1	2	3	4	5	6	7	8	9
		2				12	22	32	
		Apple				Grape	Kiwi	Fig	



HASH1(32) = 2; HASH2(32) = 1 + (32 % 9) = 1 + 5 = 6
K = 1; INDEX I = (2 + 1 * 6) % 10 = 8 % 10 = 8

Inserting 32

Insert (32, Fig)

0	1	2	3	4	5	6	7	8	9
		2				12	22	32	
		Apple				Grape	Kiwi	Fig	



HASH1(32) = 2; HASH2(32) = 1 + (32 % 9) = 1 + 5 = 6
K = 1; INDEX I = (2 + 1 * 6) % 10 = 8 % 10 = 8

Benefit of Double Hashing

0	1	2	3	4	5	6	7	8	9
		2				12	22	32	
		Apple				Grape	Kiwi	Fig	





Quadratic Probing



**Quadratic
Probing**

**Double
Hashing**